

Robertson Derivation Document — Reading Guide

Song Zhengdao

README: How to Read the Robertson Mathematical Derivation & Numerical Analysis Document

October 2026

Project Overview

This repository hosts the mathematical derivation and numerical analysis documentation for the Robertson chemical kinetics problem. The Robertson problem is a classic stiff system of ordinary differential equations (ODEs) describing the evolution of three reactant concentrations over time. Its three rate constants span roughly nine orders of magnitude, causing fast-decaying and slow-evolving modes to coexist within the same system. This coexistence of multiple time scales is the very source of stiffness, and also the central question of this project: Explicit methods are constrained by the stability requirements of the fast mode — can implicit methods achieve comparable accuracy at lower computational cost?

This document is the deliverable of Team Member 1, covering all mathematical derivations and theoretical analysis required by the first 12 items of the project brief. Every theoretical conclusion is paired with a concrete numerical experiment specification for the coding team to implement and verify.

Repository Files

Document Organization

Every chapter in the document strictly follows a four-part structure, ensuring that each theoretical conclusion has a corresponding numerical experiment:

1. Theory: Mathematical derivations starting from first principles — rather than citing textbook formulas, results are rebuilt from fundamental tools such as Taylor expansions.
2. Prediction: Concrete, testable predictions derived from the theory, intended for verification through numerical experiments.

3. Required Experiments: A clear specification of the numerical experiments the coding team must implement, including function interfaces and output requirements.
4. Explanation Notes: How to interpret the experimental results, common pitfalls, and connections to subsequent chapters.

Reading tip: If you only care about "what to implement", jump straight to the "Required Experiments" subsection of each chapter. If you need to understand "why it is designed this way", read the "Theory" subsection in order.

Document Structure & Chapter Overview

The document consists of 12 chapters, organized in a logical progression: "Model formulation → Method derivation → Stability analysis → Stiffness analysis → Experiment design". The table below lists each chapter's topic and core content:

Suggested Reading Paths

Team members in different roles can follow the paths below to retrieve the information they need most efficiently:

Coding Team (Recommended Path)

1. Read §1 first to understand the Robertson ODE system and the two diagnostics: conservation and positivity.

2. Read §2 to learn the formulas and theoretical convergence orders of the three methods, and to settle on a unified interface design.
3. Read §3 to grasp the Newton iteration implementation details for each Implicit Euler step (linear solve, tolerance settings).
4. Read §4 to clarify the experimental criteria for convergence tests (four halving step sizes, observed-order computation).
5. Read §6 and §8 to implement the Jacobian function and stiffness ratio computation.
6. Read §10 to implement the four core diagnostics and the pass/fail rules.
7. Read §11 and §12 to design the matched-error comparison and Newton tolerance sweep experiments.

Shortcut: The "Required Experiments" subsection of each chapter already lists function interfaces and output requirements — it can serve directly as an implementation spec.

Theory Team (Recommended Path)

1. Follow §1 → §2 → §5 to understand the connection between the model, methods, and stability theory.
2. Read §6 → §7 → §8 to understand the Jacobian structure, the origin of the zero eigenvalue, and stiffness analysis.
3. Read §9 for the quantitative estimate of how Explicit Euler is constrained by stiffness.
4. Read §4 and §11 for the methodology of convergence theory and fair comparison.

Quick Overview (Recommended Path)

1. Read §1.1 and §1.2 for the background of the Robertson problem and the conservation law.
2. Read the §2.4 comparison table of the three methods for a quick grasp of their characteristics.
3. Read §5.5 to understand why A-stability is crucial for stiff systems.
4. Read §10.2 and §10.3 for the four core diagnostics and the pass/fail rules.

Prerequisites

The following background is recommended for reading this document:

- Ordinary Differential Equations: initial value problems, existence and uniqueness, Lipschitz condition.
- Numerical Analysis Fundamentals: Taylor expansion, local truncation error vs. global error, explicit vs. implicit methods.
- Linear Algebra: matrix eigenvalues, Jacobian matrix, LU factorization and linear solves.
- Numerical Stability Concepts: absolute stability region, A-stability, and the basic meaning of stiffness.
- Newton's Method: linearization of nonlinear equations, convergence conditions.

Relationship Between This Document and the Code

This document covers the mathematical derivation and theoretical analysis only — it does not contain code. The "Required Experiments" subsection of each chapter provides the coding team with a clear implementation spec, including:

- Function names and interfaces to implement (e.g., `robertson_jacobian(t, y)`, Newton solver, etc.).
- Output quantities to record (e.g., residual, iteration count, converged/failed status, max error).
- Experiment design parameters (e.g., step-size sweep range, tolerance sweep sequence, four halving step sizes).
- Verification criteria (e.g., whether the observed order matches the theoretical order, whether conservation and positivity hold).

If the coding team has questions about a derivation during implementation, they can trace back to the "Theory" subsection of the corresponding chapter for the full derivation. The "Explanation Notes" subsection also highlights common pitfalls and things to watch for — it is recommended to consult it when analyzing experimental results.

Key Terminology Quick Reference

Contact & Feedback

If you have questions about the document, find a derivation error, or would like a theoretical detail expanded, please submit a GitHub Issue and reference the relevant chapter number (e.g., §6.2) for quick triage.